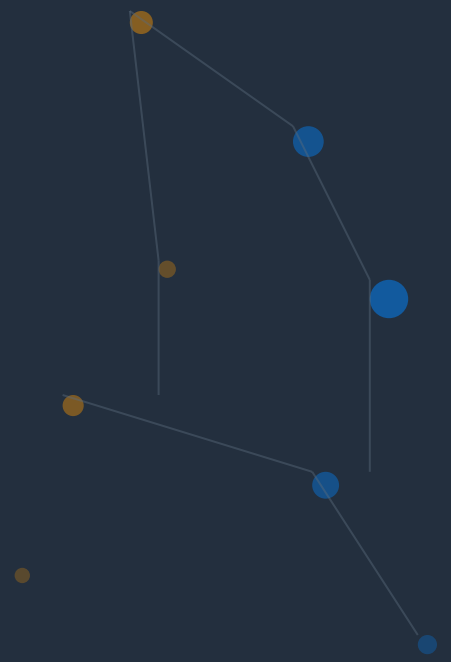
Package Management

RPM과 DNF로 소프트웨어 패키지를 설치·업데이트·삭제하기



1

Package Management

●패키지(package)란 하나의 소프트웨어 애플리케이션을 구성하는 파일과 메타데이터의 묶음

●설치·유지보수·배포를 쉽게 하도록 각 리눅스 배포판에 맞는 형식으로 포장되어 있음

Package 예시

`httpd` `mysql-server` `openssh-server` `git` `python`

Package Management Tools

●패키지의 설치·업데이트·삭제를 도와주는 패키지 관리 도구를 학습

●본 과정은 Red Hat 계열의 RPM(Red Hat Package Manager)과 DNF(Dandified YUM)에 초점을 둠

배포판 계열	패키지 포맷	관리 도구
Red Hat 계열 (RHEL·Fedora·CentOS)	`.rpm`	**rpm → yum → dnf**
Debian 계열 (Debian·Ubuntu)	`.deb`	**dpkg → apt / apt-get**

1-1

RPM (Red Hat Package Manager)



RPM Package Management

●RPM 패키지(.rpm)를 설치·관리하기 위해 만들어진 기본 도구

●패키지에는 소프트웨어 파일뿐 아니라 메타데이터가 함께 포함됨

메타데이터

●버전 번호(version), 패키지에 포함된 파일 목록

●패키지 설명(description), 패키저(packager) 정보

●다른 패키지와의 의존성(dependencies) 정보

신뢰성

RPM 패키지는 배포자가 GPG 키로 전자 서명하여 무결성과 출처를 보장합니다.

RPM 명령어

옵션

- -i : install (설치)　　-U : upgrade (업그레이드)
- -e : erase (삭제)　　-q : query (조회)
- -v : verbose (상세 출력)　　-h : hash (진행률 #표시)

install · upgrade

```
$ rpm -Uvh mypkg-2.7-5.x86_64.rpm
$ rpm -ivh mypkg-2.7-5.x86_64.rpm
# 설치(install) / 업그레이드(upgrade)
```

erase · query

```
$ rpm -e mypkg
$ rpm -q mypkg
# 삭제(erase) / 존재 여부 조회(query)
```

파일명 규칙

mypkg-2.7-5.x86_64.rpm → 이름(mypkg) · 버전(2.7) · 릴리스(5) · 아키텍처(x86_64)

RPM의 단점

- 매우 원시적인(primitive) 도구 — 저수준 설치/삭제만 담당
- 소프트웨어 의존성(dependency)을 사용자가 직접 해결해야 함

dependency error

```
$ rpm -ivh mypkg-2.7-5.x86_64.rpm
error: Failed dependencies:
        libssl.so.10()(64bit) is needed by mypkg-2.7-5.x86_64
        libcurl >= 7.29 is needed by mypkg-2.7-5.x86_64

# 필요한 라이브러리를 사용자가 하나씩 직접 설치해야 함
# → 이 문제를 해결하기 위해 등장한 것이 YUM, 오늘날엔 후속인 DNF를 사용
```

1-2

DNF (Dandified YUM)

●RPM을 기반으로 더 쉽고 편리한 패키지 관리를 제공하는 도구

●Red Hat 공식 저장소는 물론 서드파티 저장소에서도 패키지를 설치·삭제·조회·관리

●저장소(repository) 정보는 /etc/yum.repos.d/ 에 저장됨 (하위 호환을 위해 경로명은 yum 시절 그대로 사용)

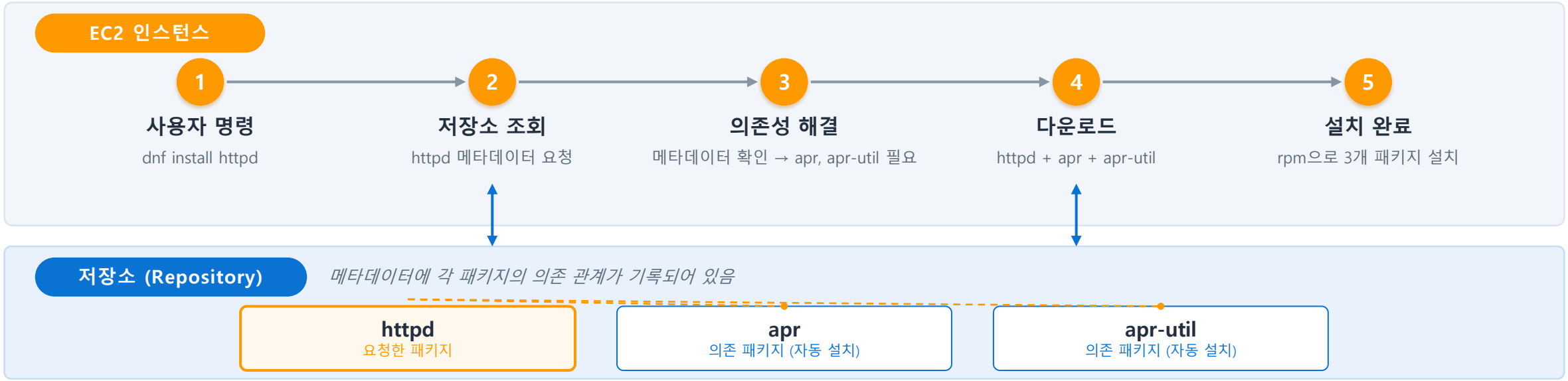
●설치·업데이트·삭제 시 의존성을 자동으로 해결함



TIP

YUM은 DNF의 이전 세대 도구입니다. RHEL 7·CentOS 7 등 구버전 시스템은 여전히 yum 명령을 사용하며, 명령 문법은 dnf와 거의 동일하게 호환됩니다.

DNF 동작 흐름 (예: httpd 설치)



핵심

- 저장소(repository): 네트워크로 접근 가능한 패키지들의 모음
- DNF는 저장소 메타데이터를 참고해 필요한 의존 패키지까지 자동으로 함께 다운로드·설치함

저장소 설정 파일 예시

●/etc/yum.repos.d/ 안에는 .repo 확장자의 텍스트 파일들이 있고, 파일마다 저장소 하나의 정보를 저장

●EC2(Amazon Linux 2023)에는 기본적으로 amazonlinux.repo 파일이 AWS 공식 저장소 주소를 지정

amazonlinux.repo

```
$ cat /etc/yum.repos.d/amazonlinux.repo
[amazonlinux]
name=Amazon Linux 2023 repository
mirrorlist=https://cdn.amazonlinux.com/al2023/core/mirrors/...
gpgcheck=1
enabled=1
# name=설명 · mirrorlist=서버 주소 · gpgcheck=서명 검증 여부 · enabled=사용 여부(1/0)
```

1) Checking for Updates

dnf check-update

● 로컬 시스템에 설치된 패키지 중 업데이트가 가능한 항목을 확인

check-update

```
$ dnf check-update

Package name           Version                Repo
httpd.x86_64           2.4.62-1.amzn2023.0.1    amazonlinux
git.x86_64             2.47.1-1.amzn2023.0.1    amazonlinux
kpatch-runtime.x86_64 0.9.9-2.amzn2023       kernel-livepatch
```

amazonlinux → AL2023 기본 저장소 · kernel-livepatch → 라이브패치 전용 저장소

설치된 패키지 업데이트하기

●로컬 시스템에 설치된 모든 패키지를 최신 버전으로 업데이트

update all packages

```
$ sudo dnf update
```

⚠ 이 명령은 관리자 권한(superuser)이 필요합니다

2) Displaying Package Info

dnf info package_name

●지정한 패키지의 상세 정보를 표시 — 설치되어 있으면 로컬에 설치된 버전 정보를, 설치되어 있지 않으면 저장소에서 설치 가능한 버전 정보를 표시

dnf info

```
$ dnf info httpd

Name         : httpd
Version      : 2.4.62
Release      : 1.amzn2023.0.1
Size         : 1.7 M
Repo         : @amazonlinux (설치됨)
Summary      : Apache HTTP Server
```

3) Updating Packages

dnf update package_name

●지정한 패키지 하나만 최신 버전으로 업데이트

update single package

```
$ sudo dnf update httpd

Resolving dependencies...
httpd.x86_64 0:2.4.57-1.amzn2023.0.1 will be updated
httpd.x86_64 0:2.4.62-1.amzn2023.0.1 will be an update
Complete!
```

4) Listing Packages

dnf list (1)

dnf list

```
$ dnf list available
# 저장소에서 설치 가능한 모든 패키지 목록



$ dnf list installed
# 현재 시스템에 설치된 패키지 목록

$ dnf list installed | wc -l
# 설치된 패키지의 총 개수 세기

$ dnf list installed | grep nginx
# nginx 설치 여부 확인
```

dnf list installed → 설치된 패키지 목록 · dnf list available → 설치 가능한 패키지 목록

dnf list (2)

●주어진 패키지 이름(package-name)과 일치하는 패키지(설치 여부 무관)를 조회

dnf list <package>

```
$ dnf list nginx
$ dnf list mysql*
# 와일드카드(*)로 이름이 비슷한 패키지도 함께 조회 가능
```

5) Searching Packages

dnf search package_name

●모든 저장소에서 package-name과 관련된 패키지를 검색

dnf search

```
$ dnf search nginx
```

dnf search

●키워드로 패키지 이름과 요약(summary)에서 검색

●정확한 패키지 이름을 모르거나 정보가 일부만 있을 때 유용

dnf list

●이름(또는 이름 패턴)으로만 조회 — 요약/설명 텍스트는 검색하지 않음

6) Installing Packages

dnf install package_name

로컬 패키지 설치

```
$ sudo dnf install /path/to/vim-enhanced-9.1.785-1.amzn2023.0.1.x86_64.rpm
# 로컬 RPM 패키지 설치
```

원격 저장소에서 설치

```
$ sudo dnf -y install [package-name]
# 원격 저장소에서 설치, -y 는 모든 프롬프트에 자동으로 yes 응답
```

대체 버전 추가 설치 (AL2023 예시)

```
$ sudo dnf install gcc14
# AL2023 기본 GCC 11과 별도로 GCC 14를 추가 설치 (gcc14 명령으로 사용)
```

7) Removing Packages

dnf remove package_name

●지정한 패키지와, 그 패키지가 있어야 동작하는(그 패키지에 의존하는) 다른 패키지들을 함께 제거

dnf remove

```
$ sudo dnf remove [package-name]
```

1-3

Install Apache HTTP Server (1)

1

Package Check

httpd 패키지 설치 여부 확인

```
$ dnf list installed | grep httpd
```

2

Package Installation

httpd 패키지 설치

```
$ sudo dnf install httpd
```

3

Service Startup

httpd 서비스 시작 및 상태 확인

```
$ sudo systemctl start httpd
$ sudo systemctl status httpd
```

Install Apache HTTP Server (2)

●Package Removal: httpd 패키지 제거

httpd 제거

```
$ sudo dnf remove httpd
```

정리

●설치 확인 → 설치 → 서비스 시작/상태 확인 → (필요 시) 제거의 흐름을 기억하세요

●다음 시간에는 Apache 웹 서버를 실제로 설치/구동하고 EC2 보안 그룹에서 접속을 허용하는 실습을 진행합니다